URL Shortener - Step 4: Wrap Up

1. What We Covered

In this chapter, we discussed the complete design of a URL shortener system:

- API Design (Shortening + Redirecting)
- Data Model (`<shortURL, longURL>`)
- Hash Functions (Hashing & Base62)
- URL Shortening Flow
- URL Redirecting Flow (Cache + DB)

2. Additional Important Talking Points

Rate Limiter

Problem:

- Malicious users may send **huge number of requests**
- Can overload system

Solution:

- Apply **rate limiting**
    - Based on:
        - IP address
        - User ID
        - API rules

✔ Prevents abuse
✔ Protects system resources

Web Server Scaling

Key Property:

- Web tier is **stateless**

Benefit:

- Easy horizontal scaling:
    - Add servers → handle more traffic
    - Remove servers → reduce cost

✔ Supports high scalability

Database Scaling

Techniques:

1. **Replication**
    - Multiple copies of database
    - Improves:
        - Availability
        - Read performance
2. **Sharding**
    - Split data across multiple databases
    - Each shard handles a subset of data

✔ Enables large-scale storage

Analytics

Why important?

- Understand user behavior:
    - Number of clicks
    - Click timing
    - Traffic sources

Use cases:

- Business insights

- Optimization
- Monitoring popularity of links

Availability, Consistency, Reliability

Core system design principles:

Availability

- System remains operational

Consistency

- Data remains correct across system

Reliability

- System performs correctly over time

✔ Critical for large-scale distributed systems

3. Final Takeaways

- URL shortener design combines:
    - Efficient encoding (Base62)
    - Fast lookup (Cache)
    - Scalable storage (DB + Sharding)
- Read-heavy optimization is essential
- Supporting components (rate limiter, analytics) improve robustness